SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 1 – Analytical Problem Solving
Weightage:	40% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	P. Sai Charan Tej	**Reg. No.:**	192465048
Section / Batch:		**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems.
- Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required.
- Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis.
- Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

Question Type	Focus Area	Questions
Application-Based Questions	Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis	Q1

SECTION A – APPLICATION BASED QUESTIONS

Q43. Selection Sort – Dataset with Zero and Repetitions

Apply the Brute Force technique using Selection Sort on the dataset [16, 0, 9, 16, 4, 0]. Clearly interpret the problem and justify the use of Selection Sort for this dataset. Perform the sorting step-by-step and show the intermediate array after each pass. Count the number of comparisons and swaps made during execution. Analyze the time complexity in best, average, and worst cases. Verify the correctness of the final sorted output and interpret the effect of repeated zero values on the sorting process.

Code of Conduct

I P. Sai Charan Tej(192465408)__(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%				
Application of Concepts	30%				
Problem-Solving Approach	20%				
Accuracy of Solution	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Assessment Tool – Selection Sort Using Brute Force

Problem Statement

Apply the Brute Force technique using Selection Sort on the dataset [16, 0, 9, 16, 4, 0]. Sort the elements in ascending order, display the array after each pass, count the total comparisons and swaps, analyze the time complexity, and explain the effect of repeated zero values.

Algorithm (Selection Sort)

1. Find the smallest element in the unsorted portion of the array.
2. Swap it with the first unsorted element.
3. Move the sorted boundary one position to the right.
4. Repeat until the array is completely sorted.

Step-by-Step Execution

Initial Array: [16, 0, 9, 16, 4, 0]

Pass	Minimum Element	Swap	Array After Pass
Initial	-	-	[16, 0, 9, 16, 4, 0]
Pass 1	0 (index 1)	16 ↔ 0	[0, 16, 9, 16, 4, 0]
Pass 2	0 (index 5)	16 ↔ 0	[0, 0, 9, 16, 4, 16]
Pass 3	4 (index 4)	9 ↔ 4	[0, 0, 4, 16, 9, 16]
Pass 4	9 (index 4)	16 ↔ 9	[0, 0, 4, 9, 16, 16]
Pass 5	16	No Swap	[0, 0, 4, 9, 16, 16]

Final Sorted Array

[0, 0, 4, 9, 16, 16]

Comparison Count

For n = 6

Comparisons = 5 + 4 + 3 + 2 + 1 = 15

Total Comparisons = 15

Swap Count

Pass 1 → 1 swap

Pass 2 → 1 swap

Pass 3 → 1 swap

Pass 4 → 1 swap

Pass 5 → No swap

Total Swaps = 4

Time Complexity Analysis

Case	Time Complexity
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$

Space Complexity: O(1)

Correctness Verification

Every pass places the smallest remaining element in its correct position. After five passes, all elements are arranged in ascending order. Therefore, the algorithm correctly sorts the dataset.

Working of Selection Sort

Selection Sort divides the array into a sorted portion and an unsorted portion. In every pass, the smallest element from the unsorted portion is selected and moved to the sorted portion. This process continues until the entire array becomes sorted.

Advantages

- Simple and easy to understand.
- Uses only constant extra memory.
- Performs fewer swaps than Bubble Sort.
- Suitable for small datasets.

Disadvantages

- Inefficient for large datasets.
- Always performs $O(n^2)$ comparisons.
- Not a stable sorting algorithm.

Effect of Repeated Zero Values

The dataset contains two zero values. Selection Sort correctly places both zeros at the beginning of the sorted array. Duplicate values do not affect correctness, although Selection Sort is not stable.

Result

The dataset [16, 0, 9, 16, 4, 0] was successfully sorted using the Selection Sort (Brute Force) algorithm.
Sorted Array: [0, 0, 4, 9, 16, 16]
Comparisons: 15
Swaps: 4
Time Complexity: $O(n^2)$
Space Complexity: O(1)

Python Code Implementation

```
# Selection Sort using Brute Force

arr = [16, 0, 9, 16, 4, 0]
n = len(arr)

comparisons = 0
swaps = 0

print("Initial Array:", arr)

for i in range(n - 1):
    min_index = i
    for j in range(i + 1, n):
        comparisons += 1
        if arr[j] < arr[min_index]:
            min_index = j
    if min_index != i:
        arr[i], arr[min_index] = arr[min_index], arr[i]
        swaps += 1
    print(f"After Pass {i + 1}: {arr}")

print("\nFinal Sorted Array:", arr)
print("Total Comparisons:", comparisons)
print("Total Swaps:", swaps)

print("\nTime Complexity:")
print("Best Case    : O(n²)")
print("Average Case : O(n²)")
print("Worst Case   : O(n²)")
print("Space Complexity: O(1)")
```

Program Output